

PySpark – Theoretical Assignment 2

1. Different File Formats

Direct Answer

File formats define how data is stored, organized, compressed, and read by data-processing systems such as PySpark. The formats covered here are Text/CSV, Avro, Parquet, and ORC.

Concept Explained in Simple Terms

Text/CSV and Avro are row-based formats. Complete records are stored together. **Parquet and ORC are columnar formats.** Values are organized by column, which is particularly useful for analytical queries that read selected columns.

Technical / Code Example

```
# Example CSV
order_id,order_date,customer_id,quantity,unit_price
0101,2026-01-01,C101,2,500
0102,2026-01-02,C102,5,200
```

Visual / Flow

Row-based → Row 1: A | B | C | D → Row 2: A | B | C | D

Columnar → Column A: A | A | A → Column B: B | B | B

Text / CSV: Row-based, simple, human-readable, and widely supported. It normally does not carry a rich embedded schema. Your classroom example treats 1 GB as approximately 1 GB without effective compression.

Avro: Row-based serialization format with schema support. Your classroom example assumes 1 GB → about 600 MB, or 40% compression. It is useful for structured data exchange/serialization and schema evolution. The schema defines expected fields and types; it should not simply be described as a general data-validation mechanism.

Parquet: Columnar format widely used for analytical workloads and data lakes. Your classroom example assumes 1 GB → about 400 MB, or roughly 55–65% compression. Its columnar layout can reduce the amount of data processed when only selected columns are required.

ORC: Columnar format designed for efficient analytical processing and commonly associated with Hadoop/Hive ecosystems. Your classroom example assumes 1 GB → about 100 MB, or roughly 90–95% compression.

Workload/cost example from your notes: Parquet example: 400 GB processed in 40 minutes → ■40 processing + ■15 storage = ■55. ORC example: 100 GB processed in 60 minutes → ■60 processing + ■10 storage = ■70. These are classroom assumptions; actual size, processing time, and cost depend on the data, compression codec, query, cluster, and workload.

Comparison

Format	Storage	Typical purpose	Example from your notes
Text/CSV	Row-based	Simple exchange/inspection	1 GB → ~1 GB
Avro	Row-based	Schema-aware serialization/exchange	1 GB → ~600 MB
Parquet	Columnar	Analytics/data lakes	1 GB → ~400 MB; 400 GB/40 min example
ORC	Columnar	Analytical/Hadoop-Hive workloads	1 GB → ~100 MB; 100 GB/60 min example

Important Points to Remember

Important: There is no universal “king” file format. Parquet and ORC are both strong analytical formats, while CSV/Text and Avro serve different needs. Compression figures are not universal benchmarks.

2. PySpark: pyspark.sql.functions and pyspark.sql.types

Direct Answer

PySpark provides modules for DataFrame functions and for defining data types and schemas.

Concept Explained in Simple Terms

pyspark.sql.types contains schema and data-type classes such as StructType, StructField, StringType, DateType, and IntegerType. **pyspark.sql.functions** provides functions and column expressions used to transform, filter, aggregate, and analyze DataFrames.

Technical / Code Example

```
from pyspark.sql import SparkSession
from pyspark.sql.types import (
    StructField, StructType, StringType,
    DateType, IntegerType
)
from pyspark.sql.functions import *

spark = SparkSession.builder \
    .appName("Retail_dat_processing") \
    .getOrCreate()

sales_schema = StructType([
    StructField("order_id", StringType()),
    StructField("order_date", DateType()),
    StructField("customer_id", StringType()),
    StructField("customer_name", StringType()),
    StructField("product_id", StringType()),
    StructField("product_name", StringType()),
```

```

    StructField("category", StringType()),
    StructField("store_id", StringType()),
    StructField("store_name", StringType()),
    StructField("city", StringType()),
    StructField("state", StringType()),
    StructField("quantity", IntegerType()),
    StructField("unit_price", IntegerType()),
    StructField("discount_pct", IntegerType())
])

input_df = spark.read.csv(
    "sales_data.csv", header=True, schema=sales_schema
)

gross_df = input_df.withColumn(
    "gross_amount",
    input_df.quantity * input_df.unit_price
)

gross_df.show(10, False)
gross_df.printSchema()
input_df.count()

```

Visual / Flow

sales_data.csv → sales_schema → input_df → withColumn()/functions → gross_df

In this complete example: SparkSession starts the application; StructType/StructField and the data-type classes define the expected schema; spark.read.csv() loads the CSV using that schema; withColumn() uses a column expression to calculate gross_amount; show() displays rows, printSchema() displays the structure, and count() returns the number of records.

Important Points to Remember

Remember: *pyspark.sql.types* = schema/data types. *pyspark.sql.functions* = DataFrame functions and column expressions.

3. What is inferSchema in PySpark? Why should it generally not be used in production?

Direct Answer

inferSchema tells Spark to automatically determine the data types of columns by examining the input data.

Concept Explained in Simple Terms

With inferSchema=True, Spark examines the input and attempts to infer types such as integer, string, or date. This is convenient during exploration, but production pipelines generally benefit from an explicitly defined schema because the expected structure and data types are controlled and predictable.

Technical / Code Example

```

# Using inference
input_df = spark.read.csv(
    "sales_data.csv",
    header=True,
    inferSchema=True
)

# Explicit production-style schema
input_df = spark.read.csv(
    "sales_data.csv",
    header=True,
    schema=sales_schema
)

```

Visual / Flow

CSV → inferSchema=True → Spark infers types
 CSV → explicit schema → developer defines expected types

Why avoid it in production? Schema inference adds schema-detection work and makes the pipeline depend on the contents of incoming data. If the source data changes unexpectedly, inferred types or behavior may not match the intended data contract. An explicit schema gives stronger control over field names, types, and expected structure.

Disadvantages: additional overhead; less predictable schema when source data changes; possible type/data-quality problems with unexpected values; less control over the production contract.

Important Points to Remember

Remember: For your sales pipeline, schema=sales_schema explicitly tells Spark what each field should be, rather than asking Spark to guess.

4. Spark Write Modes in PySpark

Direct Answer

Write modes determine what Spark does when the output destination already exists. The four primary modes are **append**, **overwrite**, **ignore**, and **error/errorifexists**.

Concept Explained in Simple Terms

append adds new data to existing output. **overwrite** replaces the existing output. **ignore** skips the write if the destination already exists. **error/errorifexists** throws an error when the destination already exists.

Technical / Code Example

```
df.write.mode("append").csv("output/sales")
df.write.mode("overwrite").csv("output/sales")
df.write.mode("ignore").csv("output/sales")
df.write.mode("error").csv("output/sales")
```

Visual / Flow

append → ADD
overwrite → REPLACE
ignore → SKIP
error → STOP

Example: If yesterday's sales output already exists, append can add today's records; overwrite can regenerate the complete output; ignore can prevent an existing destination from being changed; error can stop the job and force the situation to be handled explicitly.

Comparison

Mode	If output exists	Meaning	Typical use
append	Write succeeds	Add new data	Incremental/continuous data
overwrite	Existing output replaced	Regenerate output	Full refresh
ignore	Write is skipped	Keep existing output	Avoid changing existing result
error / errorifexists	Job fails	Require destination to be absent	Explicit safety behavior

Important Points to Remember

Remember: error/errorifexists is the default DataFrameWriter behavior when the destination already exists.

5. How does withColumn() work in PySpark?

Direct Answer

withColumn() is used to add a new column or replace an existing column in a DataFrame.

Concept Explained in Simple Terms

It takes a column name and a column expression, evaluates that expression for each row, and returns a new DataFrame. In your retail example, the expression `quantity × unit_price` calculates a gross amount for every record.

Technical / Code Example

```
gross_df = input_df.withColumn(
    "gross_amount",
    input_df.quantity * input_df.unit_price
)

# Example row:
# quantity = 2, unit_price = 500
# gross_amount = 2 * 500 = 1000

gross_df.show(10, False)
```

Visual / Flow

input_df → withColumn("gross_amount", quantity × unit_price) → gross_df

Complete logic: Spark reads the existing quantity and unit_price values for each row, evaluates the multiplication expression, places the result in the new gross_amount column, and returns a new DataFrame. If the supplied column name already exists, withColumn() replaces that column with the new expression result.

Important Points to Remember

Remember: withColumn() is for adding or redefining a column using a column expression; it returns a new DataFrame rather than modifying the original DataFrame in place.

6. Difference between withColumn() and withColumnRenamed()

Direct Answer

Both operate on DataFrame columns, but they solve different problems: **withColumn()** creates or redefines a column, while **withColumnRenamed()** changes the name of an existing column.

Concept Explained in Simple Terms

withColumn() is used when you need to calculate/add/replace a column. **withColumnRenamed()** is used when the existing data is correct and only the column name needs to change.

Technical / Code Example

```
# Add/calculated column
gross_df = input_df.withColumn(
    "gross_amount",
    input_df.quantity * input_df.unit_price
)

# Rename existing column
df = input_df.withColumnRenamed(
    "customer_name",
    "customer_full_name"
```

)

Visual / Flow

withColumn() → define/calculate column → new DataFrame

withColumnRenamed() → change column name → new DataFrame

Complete logic: withColumn() receives a new column name and an expression, such as quantity × unit_price, so the resulting column contains newly calculated values. withColumnRenamed() receives an existing name and a new name; it keeps the column's data but changes its label.

Comparison

Function	Purpose	Example	Result
withColumn()	Add or replace column	quantity × unit_price	Creates/redefines values
withColumnRenamed()	Rename existing column	customer_name → customer_full_name	Values stay the same

Important Points to Remember

Remember: withColumn() answers "What should this column contain?"; withColumnRenamed() answers "What should this column be called?"